

Solutions

1. General

ANSI-SPARC levels:

- (a) View level
- (b) Logical level
- (c) Physical level

Point allocation: 0.5 point if one level correct; 1 point if 2 levels correct

2. Conceptual data modelling

- (a) Data model: sample solution

```
ENTITIES/CLASSES
-- Note: default attribute cardinality is 1..1
```

```
ArtSection
  artType: string
Employee
  name: string
  emailAddress: string
Curator
  ISA: employee
  specialism: string
ArtWork
  collectionID: string
  artist: string
  title: string
  year: integer
```

```
RELATIONSHIPS/ASSOCIATIONS
```

```
HasCurator
  ArtSection: 1..*
  Curator: 1..*
SectionHead
  ArtSection: 0..1
  Employee: 1..1
Displays
  ArtSection: 0..1
  ArtWork: 0..* (1..* or 2..* also fine)
```

Remarks:

- Correct set of entities/classes, including attributes: 1 point

- Correct set of relationships/associations: 1 point
 - Correct specification of cardinalities: 1 point
 - Correct use of ISA relation: 1 point
 - Clear overall style, such as chosen names: 1 point
 - Errors: 0.5 point if an minor error is made, but intention is clear and correct; 0 point in other cases.
 - Different choice of vocabulary is fine, as long the intended meaning remains clear.
 - Conceptual modelling typically has some degrees of freedom (i.e. there is not one good solution). The sample solution is a kind of minimal model. You could define some attributes as entities in their own right. This is not wrong, but these entities do not really add anything. You will notice that when converting to a relational schema.
- (b) Conversion to relational schema

ArtSections(*artType*:string, *sectionHead*→*Employees*(*emailAddress*))

Employees(*emailAddress*:string, *name*:string)

Curators(*emailAddress*→*Employees*, *specialism*:string)

ArtWorks(*collectionID*:string, *artist*:string, *title*:string, *year*:integer)

HasCurator(*section*→*ArtSections*(*artType*), *curator*→*Employees*(*emailAddress*))

Displays(*section*→*ArtSections*(*artType*), *artwork*→*ArtWorks*(*collectionID*))

Remarks:

- This sample conversion has six relations, namely for all four entity sets and for two of the three relationships.
- Due to the cardinality (max 1) the *SectionHead* relation can be merged either into the *ArtSections* or the *Employees* relation. The former should be the preference, as the latter would lead to many *NULL* values.
- The *Displays* relation could be placed as an attribute in the *ArtWorks* relation. This is acceptable, but requires an explicit design decision that this attribute can take *NUL* values.

- For the ISA relation between employee and curator we use here conversion method 1 (hierarchy of tables). Method 2 is not applicable (only one subclass; no total specialisation). Method 3 (one table with null values) is possible, but this needs to be recorded as a design decision.
- The preferred notation for foreign keys is *Relation(Attribute)* in cases where the attribute name is different from the one in the table referred to. No points subtracted though in case you left the attribute in brackets out.

Points awarded: 1 point = serious effort with flaws; 1.5 point = almost correct.

- (c) Notion of “role”: indicates the role an entity/class plays in a relationship/association. This is particularly useful if the relation is between entities of the same set/class (not needed to deserve the point for this question).

3. Relational model

- (a) False
- (b) False. A superkey is a non-minimal key.
- (c) False. A foreign key may be (part of) the key of the relation it is specified in, but this is not necessarily the case.
- (d) True

4. Relational algebra

- (a) Relational algebra expression: “find the collection IDs of paintings that are being loaned to an exhibition in Rome”.

Using natural join:

$$\Pi_{colID}(\sigma_{place='Rome'}(Loans \bowtie Exhibitions))$$

Using theta-join:

$$\Pi_{Paintings.colID}(\sigma_{place='Rome'}(Loans \bowtie_{Paintings.colID=Loans.colID} Exhibitions))$$

Result is: {1, 5, 6} .

- (b) Result of cross product: $6 * 7 = 42$ tuples (1 point); 7 attributes (1 point). No duplicate attributes removed; these get names like `Paintings.colID` and `Loans.colID`.

5. SQL queries

- (a) Find paintings that have not been on loan, using NOT IN:

```
SELECT P.colID
FROM   Paintings
WHERE  P.colID NOT IN (SELECT L.colID FROM Loans L)
```

Result: {2}. You can leave out the tuple variables; it works out correctly but is less clear.

- (b) Find the places (printed as “Locations”) where paintings of Breitner have been on loan for an exhibition.

```
SELECT DISTINCT E.place AS "Locations"
FROM   Paintings P, Loans L, Exhibitions E
WHERE  P.colID = L.colID      -- join condition
      AND L.exID = E.exID    -- join condition
      AND P.artist = 'Breitner'
```

Result: {London}. Usage of DISTINCT is not strictly necessary.

- (c) Print (in alphabetical order) per artist the average insurance sum of his/her paintings. Include only artists of whom paintings have been loaned out.

```
SELECT   P.artist, AVG(L.insurance)
FROM     Paints P, Loans L
WHERE    P.colID = L.colID
GROUP BY P.artist
ORDER BY P.artist      -- ascending order is the default
```

- (d) The query prints titles of paintings twice on loan. The result: {Sick child, Prince's day}.

- (e) This query will print for all painting loans with an ensured value of more than 10 million the collection ID, the title and the ensured value. Other paintings (not loaned or with a lower insurance) will have one row with a NULL value in the column “Insurance”.

The result table will have seven rows (one row for paintings #1-#5, the first four with a NULL value for “Insurance”; two rows for painting #6).

1 point awarded per query in case of an error (either in the query or in the projected result) where the intention/method is clearly correct.

6. FD's wine tasting

```
taster, tasted-wine -> taste-notes
tasted-wine -> region
taster -> favourite-wine
```

Points: 0.5 point for one FD, 1 point for 2 FD's; 2 points for 3 FD'S,

7. Relation $R(A, B, C, D, E)$ with FDs $AB \rightarrow D$, $B \rightarrow C$, $A \rightarrow E$, and $E \rightarrow B$.

- (a) Minimal keys: all keys need to contain A , and the cover of A includes all attributes, so A is the minimal key.
- (b) The relation is not in BCNF. Violations: $AB \rightarrow D$, $B \rightarrow C$, and $E \rightarrow B$.
- (c) The relation is not in 3NF. Same violations as for BCNF.

8. Relation $R(A, B, C, D, E, F)$ with FDs $CD \rightarrow A$, $C \rightarrow B$, $D \rightarrow E$, and $B \rightarrow F$.

Key is CD (C and D missing on right-hand side, CD^+ includes all attributes).

First violation is $C \rightarrow B$. C^+ is BCF . So we create two new relations: $R_1(B, C, F)$ (key C) and $R_2(A, C, D, E)$ (old relation with BF removed; key is still CD). $R_1(B, C, F)$ has one violation, namely $B \rightarrow F$. B^+ gives simply BF , so we can split R_1 into BF and BC . Both are in BCNF (two-place relations are by definition in BCNF). $R_2(A, C, D, E)$ also has one violation: $D \rightarrow E$. D^+ is DE , so we can split R_2 into DE and ACD (old relation with E removed). Both are in BCNF, so we're done.

Result (in alphabetical order): ACD , BC , BF , and DE .

Points awarded: 1 point = serious effort with flaws; 2 points = correct method, but error in solution path.

9. Statements about database transactions:

- (a) False. Only condition is that the other transaction has not committed yet; it does not need to be aborted.
- (b) True

- (c) False. You can get an IX when another transaction has an IS. Only if the two transactions at a lower grain-size level ask for S/X lock for the same row (or set of rows) the scheduler will need to block one of them.
- (d) True

10. Conflict-serializability:

T_1 needs to be before T_3 (rw conflict for C and wr conflict for E). This is the only dependency. so the schedule is conflict-serializable.

Possible transaction orders are all that have T_1 before T_3 : (i) (T_2, T_1, T_3) , (ii) (T_1, T_2, T_3) , and (iii) (T_1, T_3, T_2) .

11. Cascading rollbacks:

- (a) T_1 aborts: T_3 needs to be aborted (wrA); this leads to T_2 also being aborted (wrB).
- (b) T_4 aborts: T_2 needs to be aborted (wrC); no more rollbacks as T_2 contains only reads.